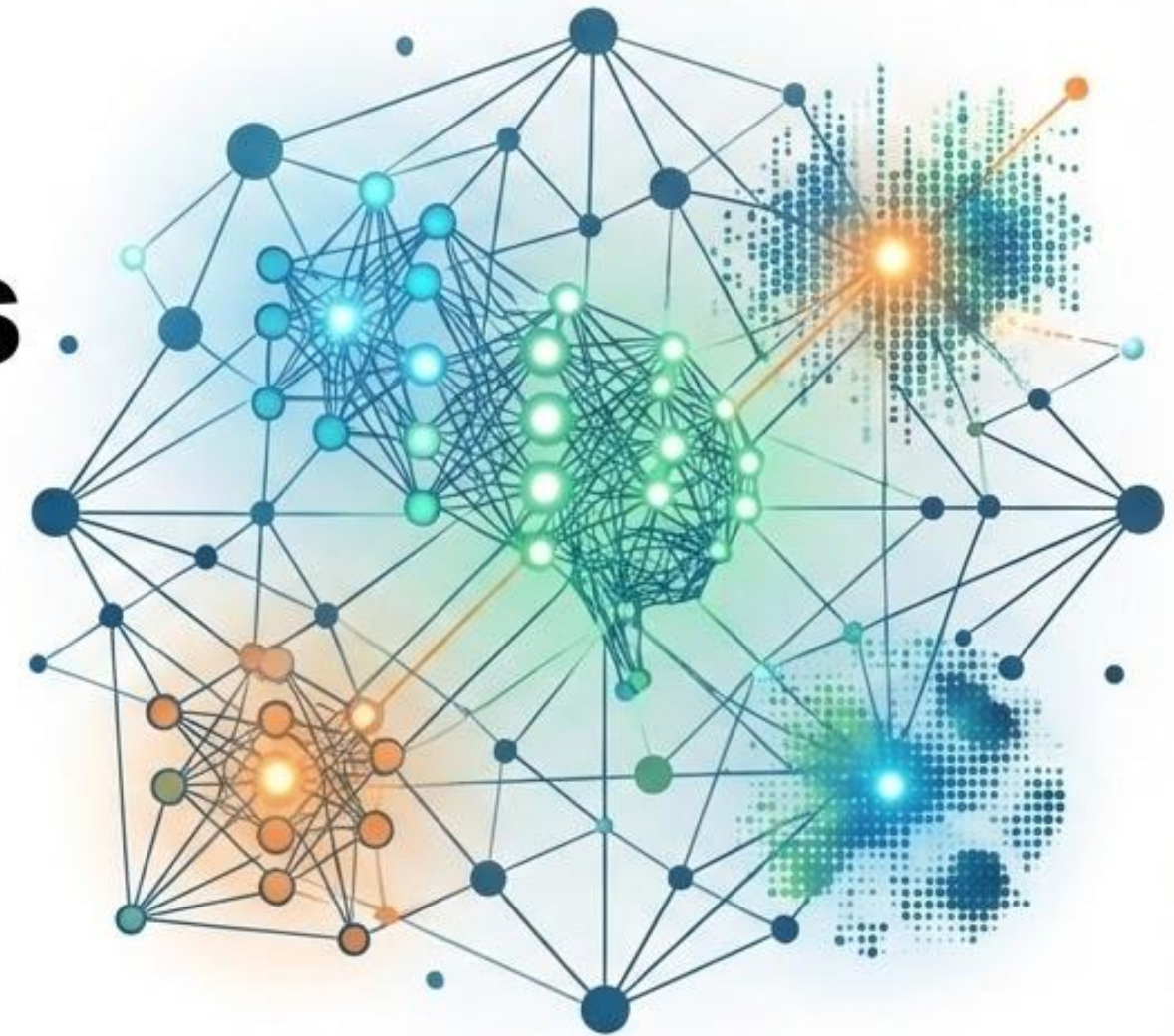


Diplomatura en python para la ciencia de datos y machine learning

Módulo 1 – Desarrollo
Avanzado en Python



6.1- Computación Científica con NumPy

Parte 1: Fundamentos: concepto, creación, indexación, slicing, copy/view, vectorización y broadcasting.

¿Qué es NumPy?

NumPy es una biblioteca de Python para trabajar con arreglos y matrices. Ofrece operaciones matemáticas rápidas de nivel C y es la base para otras bibliotecas como Pandas, SciPy y TensorFlow.



Arreglos y Matrices

Estructuras de datos eficientes.



Operaciones Rápidas

Funciones optimizadas en C.

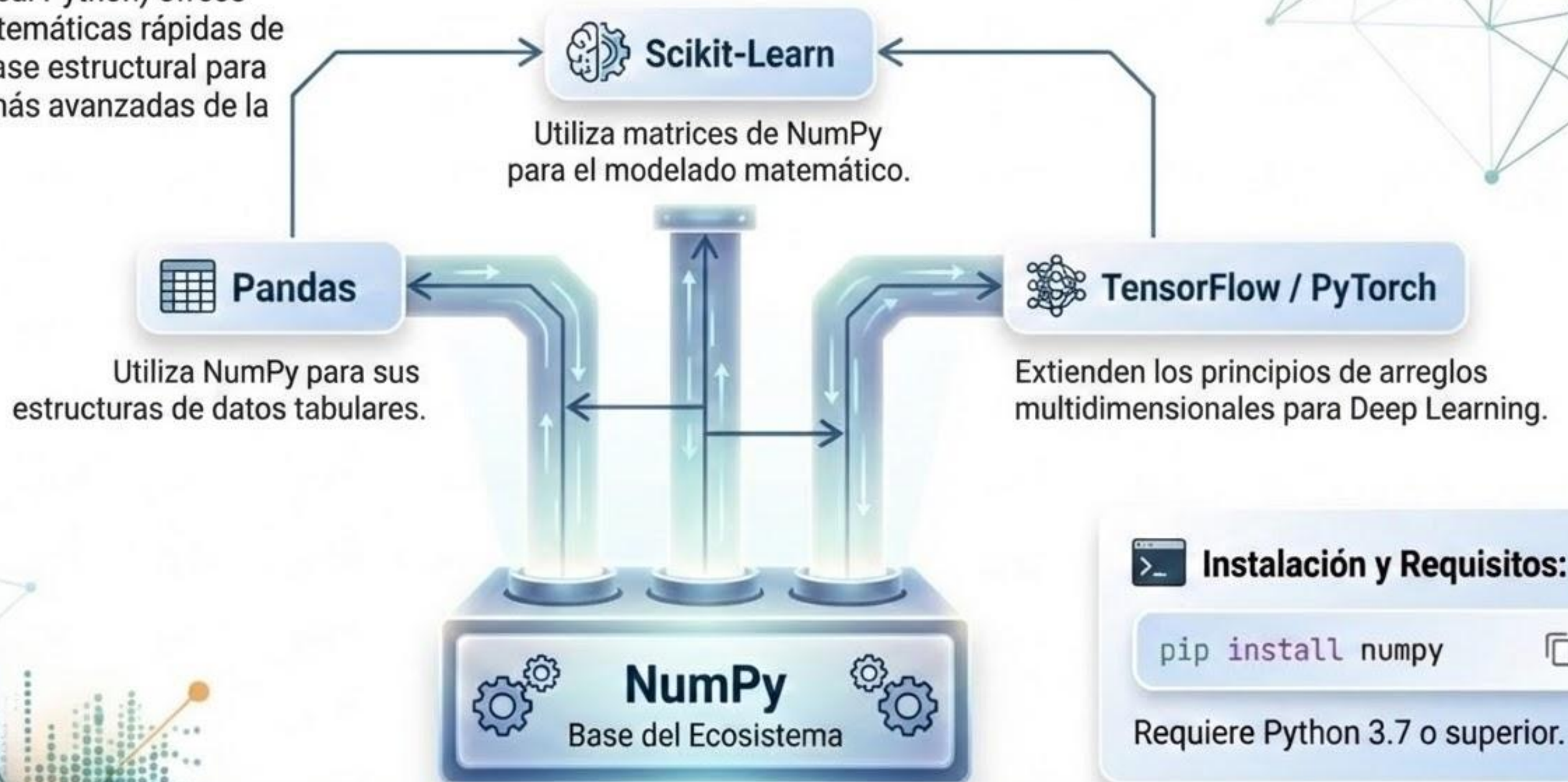


Base de Bibliotecas

Soporte para Pandas, SciPy y TensorFlow.

El motor invisible detrás del ecosistema de datos en Python

NumPy (Numerical Python) ofrece operaciones matemáticas rápidas de nivel C y es la base estructural para las bibliotecas más avanzadas de la industria.



Instalación y Requisitos:

```
pip install numpy
```

Requiere Python 3.7 o superior.

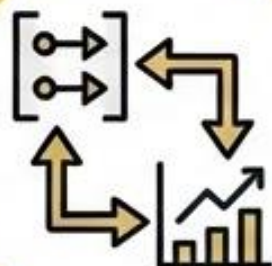
Beneficios de Usar NumPy

NumPy ofrece velocidad y eficiencia en cálculos con grandes volúmenes de datos. Soporta operaciones vectorizadas, optimiza el uso de memoria y **reduce el código** al manejar datos complejos.



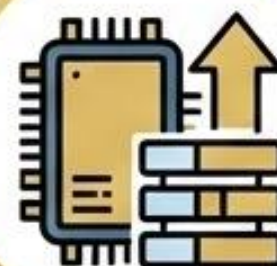
Velocidad

Cálculos rápidos con grandes datos.



Soporte Vectorizado

Operaciones optimizadas.



Optimización

Uso eficiente de memoria.



Reducción de Código

Manejo sencillo de datos complejos.

Anatomía del ndarray:

Bloques contiguos para máxima eficiencia

A diferencia de las listas dispersas de Python, un N-Dimensional Array almacena datos del mismo tipo en un **bloque contiguo** de memoria.

Creación del Array (a)

```
a = np.array([1, 2, 3],  
             dtype=np.int32)
```

Almacenamiento en Memoria

Memoria Contigua



Propiedades y Acceso

a.ndim	# 1
a.shape	# (3,)
a.size	# 3
a.nbytes	# 12

```
print(a.ndim)    # Número de dimensiones (ejes)  
print(a.shape)   # Tupla con dimensiones (ej: 3,)  
print(a.size)    # Cantidad total de elementos  
print(a.nbytes)  # Total de bytes en memoria (ej: 12)
```



Regla de Oro: Exploración Inicial

Usa siempre `.shape` y `.dtype` como primer paso para entender la estructura y el peso de tus datos antes de modelar.

Creación de Arrays en NumPy

np.array

```
arr = np.array([1, 2, 3])
```

np.zeros

```
zeros = np.zeros((2, 3))
```

np.ones

```
ones = np.ones((3, 2))
```

np.arange

```
arange = np.arange(0, 10, 2)
```

np.full

```
full = np.full((2, 2), 7)
```

np.eye

```
eye = np.eye(4)
```

np.empty

```
empty = np.empty((2, 2))
```

np.random.randint

```
randint = np.random.randint(0, 10, (2, 3))
```

np.random.randn

```
randn = np.random.randn(2, 3)
```

Comparación Formal: Listas vs. NumPy Arrays

Código de Comparación de Rendimiento

```
import time
import numpy as np

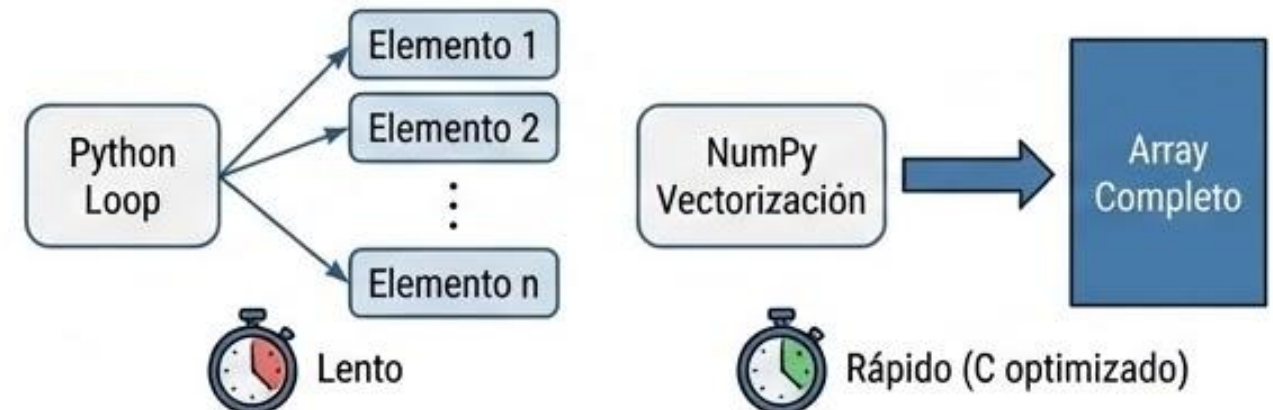
# Crear datos
lista = list(range(10_000_000))
array = np.arange(10_000_000)

# Suma con lista (Python puro)
start = time.time()
sum(lista)
tiempo_lista = time.time() - start
print(f"Tiempo lista: {tiempo_lista:.5f} s") # Ejemplo: ~0.5 s

# Suma con NumPy (vectorizada)
start = time.time()
array.sum()
tiempo_numpy = time.time() - start
print(f"Tiempo NumPy: {tiempo_numpy:.5f} s") # Ejemplo: ~0.01 s
```

Explicación del Rendimiento

-  **Loop en Python vs. Operación Vectorizada:** La suma de la lista itera elemento por elemento en Python. NumPy utiliza operaciones vectorizadas implementadas en C, procesando el array completo de forma eficiente.
-  **Overhead del Intérprete:** El bucle en Python conlleva una sobrecarga por el intérprete (comprobación de tipos, etc.) en cada iteración. Las funciones de NumPy están pre-compiladas, evitando este overhead.



Indexación y Slicing en NumPy

A) Indexación Básica

Código: `Indexación 1D`

```
import numpy as np
arr1d = np.array([10, 20, 30, 40, 50])

# Primer elemento
print(arr1d[0]) # Salida: 10
# Último elemento
print(arr1d[-1]) # Salida: 50
```

Código: `Indexación 2D`

```
arr2d = np.array([[1, 2, 3],
                  [4, 5, 6]])

# Elemento en fila 0, col 1
print(arr2d[0, 1]) # Salida: 2
# Elemento en fila 1, col 2
print(arr2d[1, 2]) # Salida: 6
```

B) Slicing (Rebanado)

Código: `Slicing 1D`

```
# [inicio:fin:paso]
arr1d = np.array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])

# Del índice 2 al 5
print(arr1d[2:6]) # Salida: [2 3 4 5]
# Con paso 2
print(arr1d[::2]) # Salida: [0 2 4 6 8]
```

Código: `Slicing 2D`

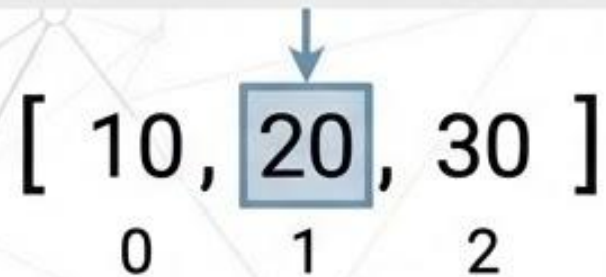
```
arr2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])

# Sub-matriz (filas 0-1, cols 1-2)
print(arr2d[:2, 1:])
# Salida: [[2 3]
#          [5 6]]
```

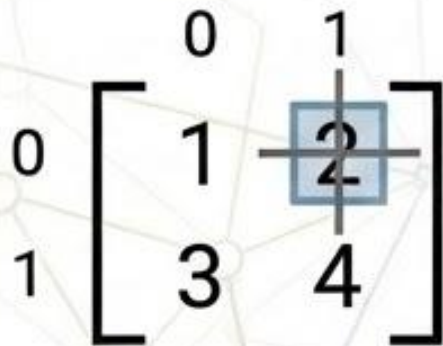
Indexado y Segmentado (Slicing) en NumPy

Indexado (Acceso a Elementos)

```
arr = np.array([10, 20, 30])  
print(arr[1]) # Resultado: 20
```


[10, 20, 30]
0 1 2

```
mat = np.array([[1, 2], [3, 4]])  
print(mat[0, 1]) # Resultado: 2
```

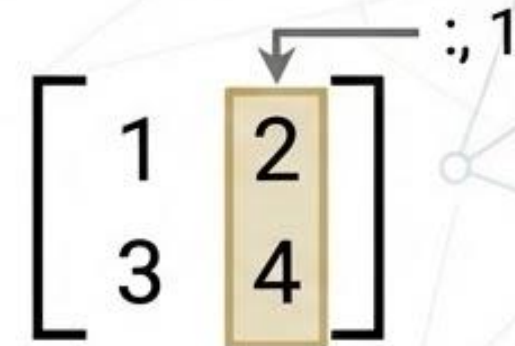

0 1
0 [1 2]
1 [3 4]

Segmentado (Slicing - Rangos)

```
arr[0:2] # Resultado: [10, 20]
```


0:2
[10, 20, 30]

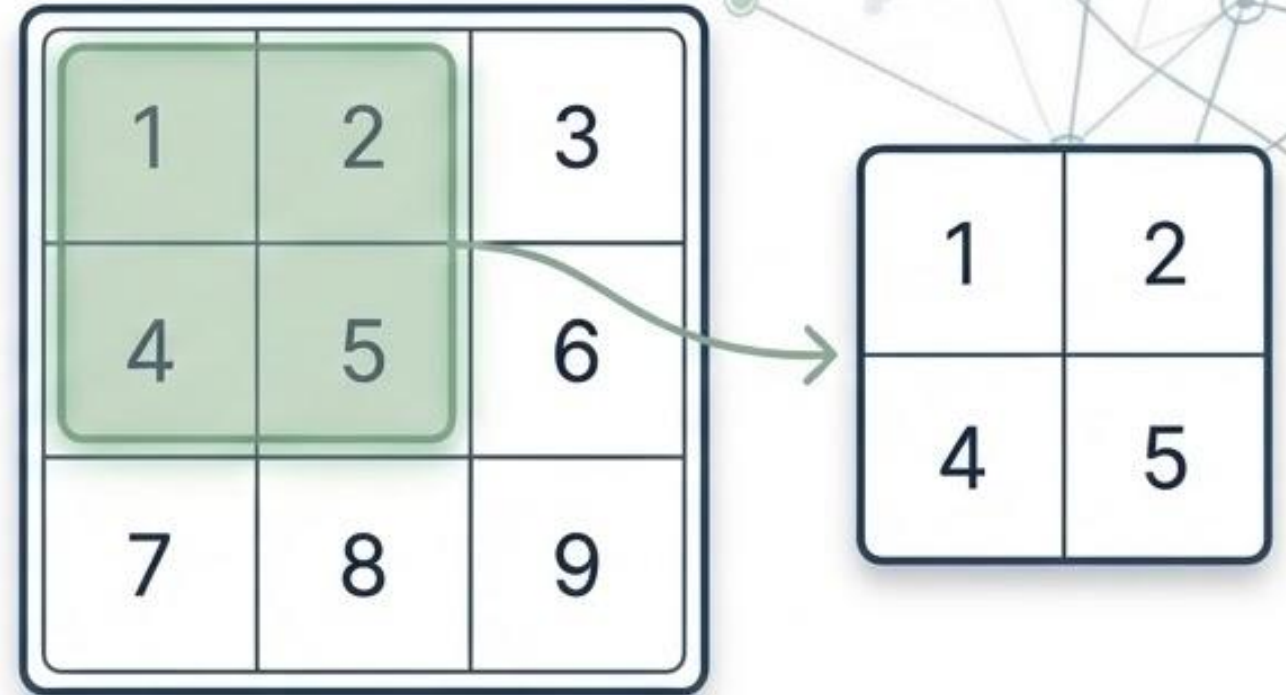
```
mat[:, 1] # Resultado: [2, 4] (Columna 1)
```


[:, 1]
[1 2]
[3 4]

Nota: Slicing crea una vista, no una copia.

Esculpiendo los datos: Indexación avanzada y Slicing

```
arr = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])  
  
# Extracción de sub-matriz (filas 0-1, columnas 0-1)  
sub_matrix = arr[0:2, 0:2]
```



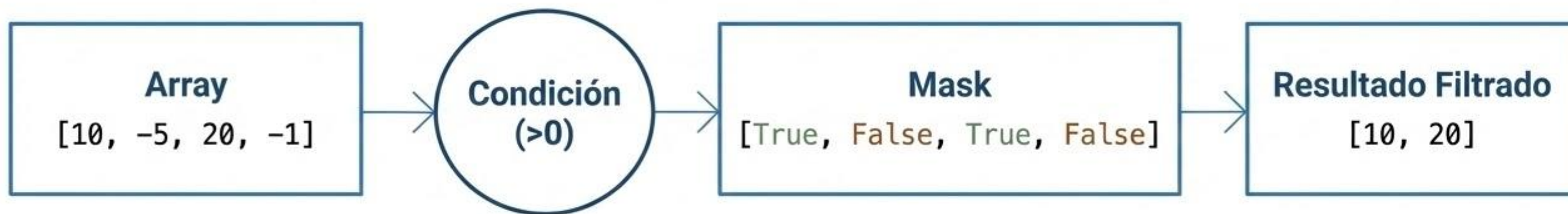
Vistas vs. Copias

¡Importante! El slicing en NumPy devuelve una vista del arreglo original para ahorrar memoria, no una copia nueva. Si modificas `sub_matrix`, alterarás `arr`. Usa `arr[0:2, 0:2].copy()` si requieres independencia total.

Máscaras Booleanas: Filtrado quirúrgico sin bucles

Las máscaras booleanas son una forma de indexación avanzada fundamental para el **análisis y limpieza de datos**.

Las máscaras generan un vector de valores de "Verdad" (True/False) que indica qué elementos mantener, **constituyendo** la base del preprocesamiento y limpieza.

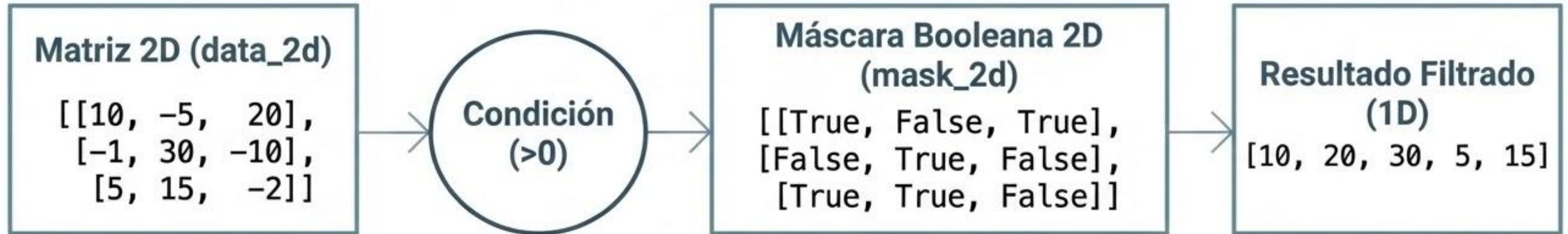


```
data = np.array([10, -5, 20, -1, 30])
mask = data > 0
solo_positivos = data[mask] # Resultado: [10, 20, 30]

# Alterando la realidad (Limpieza rápida):
data[data < 0] = 0 # Reemplaza negativos por ceros
```

Máscaras Booleanas en Matrices (2D): Filtrado

El concepto de máscaras booleanas se extiende directamente a arrays multidimensionales. La máscara resultante conserva la forma de la matriz original, y la selección produce un **array 1D aplanado** con los elementos que cumplen la condición.



```
data_2d = np.array([[10, -5, 20],  
                  [-1, 30, -10],  
                  [5, 15, -2]])  
mask_2d = data_2d > 0 # Crea una matriz booleana 3x3  
resultado = data_2d > 0 # Crea una matriz booleana 3x3  
resultado = data_2d[mask_2d] # Resultado aplanado: [10, 20, 30, 5, 15]
```

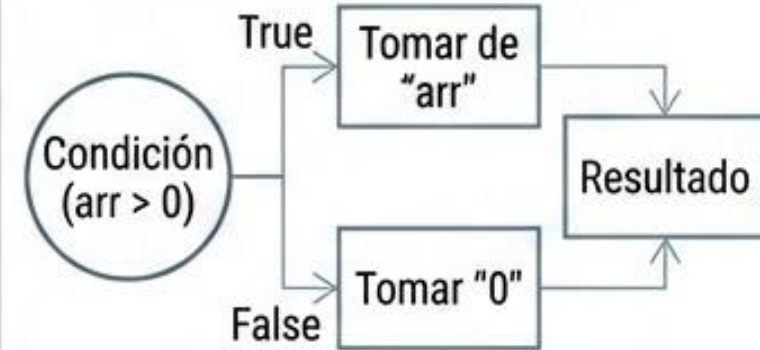
Uso de np.where: Selección Condicional y Búsqueda de Índices

La función np.where es una herramienta versátil en NumPy. Su uso más común es para seleccionar elementos de dos arrays diferentes basándose en una condición booleana, actuando como una operación vectorizada 'if-else'. También puede utilizarse para encontrar los índices de los elementos que cumplen una condición.

```
import numpy as np
arr = np.array([10, -5, 20, -1, 30])

# 1. Uso con 3 argumentos (condición, valor_si_true, valor_si_false)
# Reemplazar valores negativos con 0, mantener los positivos
resultado_condicional = np.where(arr > 0, arr, 0)
print(resultado_condicional)
# Salida esperada: [10  0 20  0 30]
```

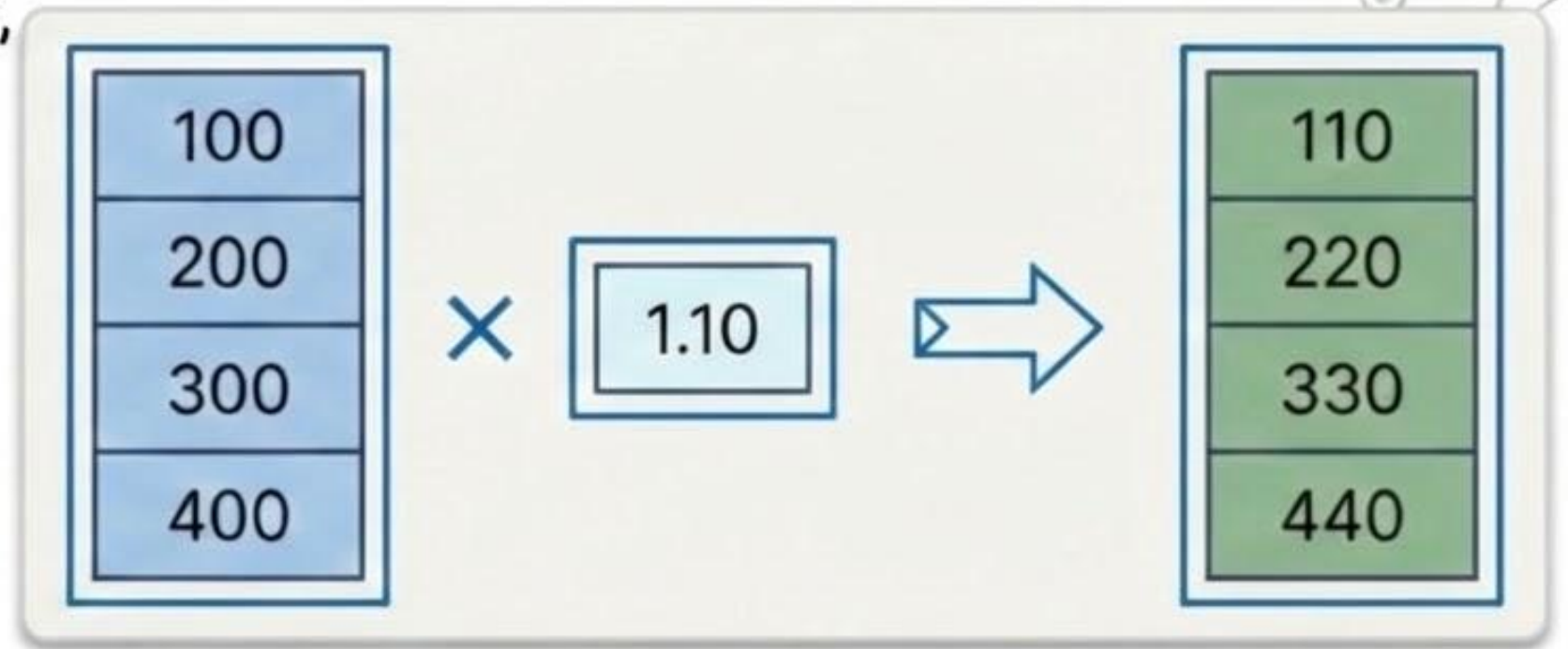
```
# 2. Uso con 1 argumento (condición)
# Obtener los índices de los elementos mayores que 10
indices = np.where(arr > 10)
print(indices)
# Salida esperada: (array([2, 4]),)  <- Devuelve tupla de índices
```



Vectorización: Pensar en bloques y abandonar los bucles for

En modelado predictivo, no trabajamos elemento por elemento. Operamos sobre estructuras de datos completas a la vez, garantizando coherencia matemática.

```
salarios = np.array([100, 200, 300, 400])  
nuevos_salarios = salarios * 1.10
```



Procesamiento Simultáneo



Tip de Eficiencia:

Si te encuentras escribiendo un bucle for sobre un arreglo de NumPy, detente. Casi siempre existe una operación vectorizada (Ufunc) más eficiente.

Broadcasting: Explicación Geométrica

```
import numpy as np

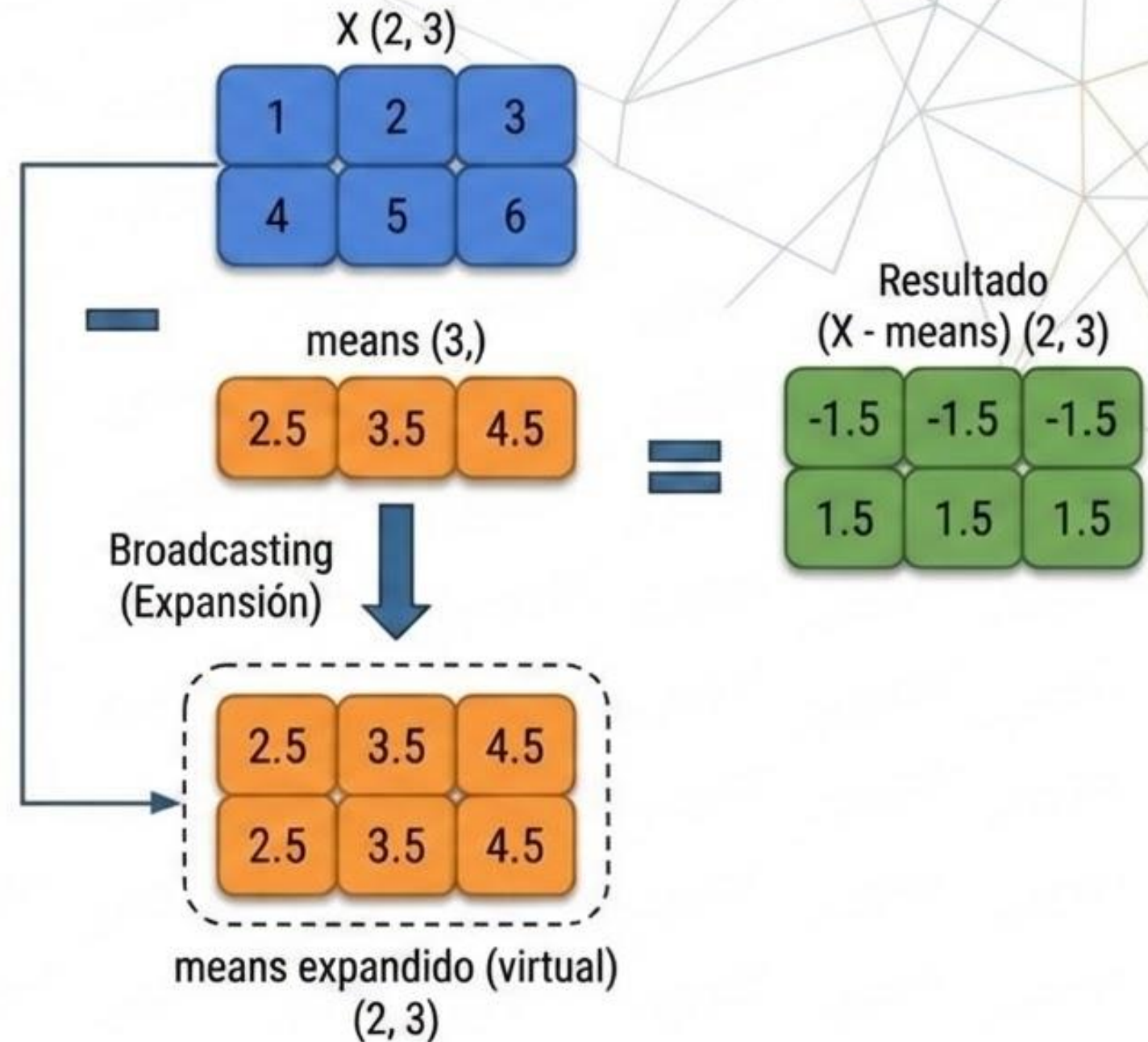
X = np.array([[1, 2, 3],
              [4, 5, 6]])

# Media por columnas (axis=0)
means = X.mean(axis=0) # [2.5, 3.5, 4.5]

# Operación con broadcasting
result = X - means
```

Conceptos Clave

- **Expansión Implícita:** NumPy "estira" virtualmente el array más pequeño (means) para igualar la forma del más grande (X) sin copiar datos en memoria.
- **Alineamiento de Dimensiones:** Las formas se comparan desde la derecha.
 - X: (2, 3)
 - means: (3,) → se trata como (1, 3)
 - Las dimensiones (3) coinciden. La dimensión 1 de means se expande a 2.



Errores Típicos: Broadcasting Inesperado

Código: El Problema

```
import numpy as np

a = np.array([1, 2, 3]) # Forma: (3,)
b = np.array([[1],
              [2],
              [3]])      # Forma: (3, 1)
```

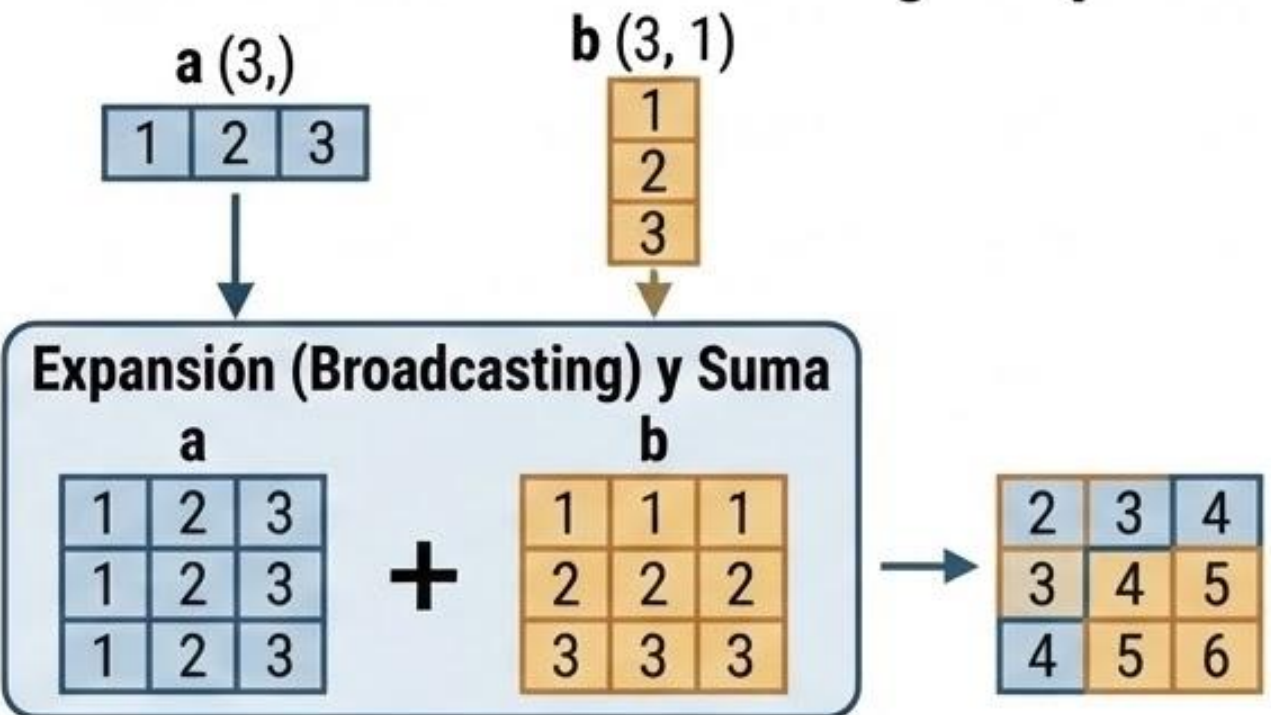
```
result = a + b
print(result)
print(result.shape)
```

```
[[2 3 4]
 [3 4 5]
 [4 5 6]]
(3, 3)
```



¡Cuidado! El resultado es una matriz (3,3), no lo que quizás esperabas.

Visualización del Broadcasting Inesperado



Ambos arrays se expanden para coincidir en forma (3, 3) antes de la suma.



¡Importante! Verifica siempre las dimensiones. El broadcasting puede crear matrices grandes inesperadamente si las formas no están alineadas como piensas.

Resumen de Ufuncs (Funciones Universales) en NumPy

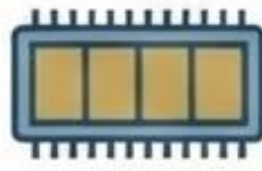
¿Qué son las Ufuncs?

Funciones rápidas que operan elemento por elemento en arrays, implementadas en C y optimizadas para la vectorización. Evitan bucles lentos de Python.

~~for x, y in
zip(arr1, arr2):
...~~



np.add(arr1, arr2)



Ventajas:

- Velocidad extrema,
- Sintaxis concisa,
- Manejo eficiente de memoria.

Ejemplos de Ufuncs Comunes



Aritméticas:

np.add (+)
np.subtract (-)
np.multiply (*)
np.divide (/)
np.power (**)



Trigonómicas:

np.sin
np.cos
np.tan
np.arcsin



Exponenciales/Log:

np.exp np.log
np.log10 np.sqrt



Comparación:

np.greater (>)
np.less (<)
np.equal (==)

Recurso: Documentación Oficial



Explora la Documentación Completa de Ufuncs

<https://numpy.org/doc/stable/reference/ufuncs.html>

Referencia exhaustiva para todas las funciones universales disponibles, comportamiento y detalles.